

Industrial Internship Report on “STUDENT MANAGEMENT SYSTEM”

Prepared by

HIRDAY SINGH

Python Learning Internship

Organized by

UPSKILL CAMPUS

The IoT ACADEMY

&

UNICONVERGE TECHNOLOGIES PVT. LTD.

Academic Year

2026-27

Executive Summary

This report presents the work completed during a four-week Python Learning Internship conducted through Upskill Campus, The IoT Academy, and UniConverge Technologies Pvt. Ltd. (UCT).

The internship focused on developing fundamental Python programming skills and applying the concepts learned through a practical project. During the training, I learned Python fundamentals including conditional statements, loops, functions, data structures, exception handling, and file handling. The internship also provided an introduction to NumPy and Pandas for numerical and data-processing operations.

As part of the project work, I developed a **Python-Based Student Management System**. The application is a console-based system designed to manage student information efficiently. It provides features for adding, viewing, searching, updating, and deleting student records. It also includes grade calculation and student performance analysis.

The application uses JSON for persistent storage of student records. Pandas is used to organize and process student information in a DataFrame, while NumPy is used for numerical calculations such as mean, median, minimum, maximum, and standard deviation.

The project provided practical experience in Python programming, data handling, input validation, exception handling, basic statistical analysis, and version control using Git and GitHub.

Overall, the internship provided useful practical exposure to Python development and helped in strengthening programming, problem-solving, and software development skills.

Table of Contents

1. Preface.....	3
2. Introduction.....	4
2.1 About UniConverge Technologies Pvt. Ltd.	4
2.2 About Upskill Campus	4
2.3 About The IoT Academy	5
2.4 Objectives of the Internship.....	5
2.5 References	6
2.6 Glossary	6
3. Problem Statement.....	7
3.1 Problem Statement.....	7
4. Existing and Proposed Solution	8
4.1 Existing Solution	8
4.2 Proposed Solution	8
4.3 Value Addition	9
4.4 Code Submission	9
4.5 Report Submission	9
5. Proposed Design / Model.....	10
5.1 High Level Diagram.....	10
5.2 Low Level Diagram.....	11
5.3 Interfaces	12
5.4 Student Record Structure	14
5.5 Data Storage	15
5.6 Performance Analysis Module.....	15
5.7 Grade Calculation.....	17
5.8 Input Validation and Exception Handling.....	17
6. Performance Test	18
6.1 Test Plan / Test Cases	18
6.2 Test Procedure.....	19
6.3 Performance Outcome.....	19
7. My Learnings	20
8. Future Work Scope	21

1. Preface

This report presents the work completed during my four-week Python Learning Internship conducted through Upskill Campus, The IoT Academy, and UniConverge Technologies Pvt. Ltd. (UCT).

The internship provided an opportunity to develop practical knowledge of Python programming and apply the concepts learned during the training to a functional project.

During the internship, I studied fundamental Python programming concepts such as conditional statements, loops, functions, data structures, exception handling, and file handling. I was also introduced to NumPy and Pandas for numerical and tabular data processing.

As the project component of the internship, I developed a Python-based Student Management System. The application provides functionality for adding, viewing, searching, updating, and deleting student records. It also includes automatic grade calculation and a performance analysis module using NumPy and Pandas. JSON is used for persistent storage of student records.

The project allowed me to apply the concepts learned during the internship in a practical manner and gain experience in organizing code into functions, validating input, handling errors, storing data, and analyzing information.

The internship also introduced me to basic software development practices such as using Git and GitHub for source-code management and maintaining different stages of development.

Overall, the internship was a valuable learning experience that helped strengthen my programming and problem-solving skills and provided a foundation for further learning in Python, data analysis, and software development.

I would like to express my gratitude to the internship coordinators, mentors, faculty members, and everyone involved in providing guidance and support throughout the internship.

2. Introduction

2.1 About UniConverge Technologies Pvt. Ltd.

UniConverge Technologies Pvt. Ltd. (UCT) is a technology company established in 2013 that works in the field of digital transformation and provides technology-based industrial solutions.

The company works with technologies such as Internet of Things (IoT), Cloud Computing, Machine Learning, Cyber Security, Python, Java, communication technologies, and front-end development.

UCT develops solutions for industrial and business applications, including IoT platforms, smart factory solutions, predictive maintenance, and data-driven monitoring systems.

The organization also focuses on applying modern technologies to solve practical industrial problems and improve operational efficiency.

During the internship program, UCT served as the industrial partner and provided the overall learning and project framework through the internship program.

2.2 About Upskill Campus

Upskill Campus is a career development and learning platform focused on helping students and learners develop industry-relevant technical and professional skills.

As part of the internship program, Upskill Campus facilitated the learning process and provided access to learning resources, project activities, and internship-related guidance.

The Python internship provided structured learning material covering fundamental Python programming concepts and introduced additional libraries such as NumPy and Pandas.

The internship also encouraged practical application of the concepts learned through the development of a Python-based Student Management System.

2.3 About The IoT Academy

The IoT Academy is an educational initiative associated with UniConverge Technologies Pvt. Ltd. that provides technical training and certification programs in various technology domains.

It focuses on providing learners with industry-oriented knowledge and practical exposure through structured training programs.

As part of this internship, The IoT Academy was involved in facilitating the learning and internship process along with Upskill Campus and UniConverge Technologies Pvt. Ltd.

2.4 Objectives of the Internship

The major objectives of the four-week Python internship were:

- To develop a basic understanding of Python programming.
 - To understand conditional statements, loops, functions, and data structures.
 - To learn basic exception handling and file handling.
 - To gain an introduction to NumPy and Pandas.
 - To apply Python programming concepts to a practical project.
 - To develop problem-solving and logical thinking skills.
 - To gain experience in organizing and maintaining source code.
 - To understand basic Git and GitHub version-control practices.
 - To gain practical exposure to software development through project implementation.
-

2.5 References

1. Python Documentation — Python programming language reference and documentation.
 2. NumPy Documentation — Reference for numerical operations and statistical functions.
 3. Pandas Documentation — Reference for DataFrame and data-processing operations.
 4. Git Documentation — Reference for version control and Git commands.
 5. GitHub Documentation — Reference for repository and source-code management.
-

2.6 Glossary

Term	Description
Python	A high-level programming language used to develop the project.
JSON	A lightweight format used for storing structured student data.
NumPy	A Python library used for numerical and mathematical operations.
Pandas	A Python library used for data manipulation and analysis.
DataFrame	A two-dimensional tabular data structure provided by Pandas.
CLI	Command Line Interface used to interact with the application through the terminal.
CRUD	Create, Read, Update, and Delete operations performed on data.
Git	A distributed version control system used to track changes in source code.
GitHub	An online platform used to host and manage Git repositories.
IDE	Integrated Development Environment used for writing and managing code.
Term	Description

3. Problem Statement

Educational institutions handle a large amount of student information, including student identification details, age, course, academic year, and marks. Managing these records manually can become time-consuming and may lead to errors, duplication, or difficulty in searching and updating information.

A simple and efficient system is therefore required to manage student records in a structured manner. The system should allow users to add new student records, view existing records, search for a particular student, update student information, and delete records when required.

In addition to basic record management, academic performance can be analyzed to provide useful information such as average marks, median marks, highest and lowest marks, standard deviation, and the number of passed and failed students.

The problem addressed in this project is to develop a **console-based Student Management System using Python** that provides basic student record management along with academic performance analysis.

The system uses **JSON** for persistent storage of student records and **Pandas and NumPy** for data processing and numerical analysis.

3.1 Problem Statement

1. The system should allow users to add student records.
2. The system should display all stored student records.
3. The system should allow students to be searched using their Student ID.
4. The system should allow individual student fields to be updated.
5. The system should allow complete student information to be updated when required.
6. The system should allow student records to be deleted.
7. The system should calculate grades based on marks.
8. The system should perform basic student performance analysis.
9. The system should validate user input and handle invalid values.
10. Student data should remain stored after the program is closed.

4. Existing and Proposed Solution

4.1 Existing Solution

Traditional student record management may rely on manual registers, spreadsheets, or basic record-keeping systems. While these methods can be sufficient for a small number of records, they can become difficult to maintain as the number of students increases.

Manual approaches may require additional effort when searching for a particular student, modifying existing information, calculating grades, or analyzing academic performance. There may also be a greater possibility of data-entry errors and inconsistent records.

Basic spreadsheet-based approaches provide better organization than manual records but may still require users to perform calculations and filtering manually.

4.2 Proposed Solution

The proposed solution is a **Python-based Student Management System** that provides a simple menu-driven interface for managing student information.

The application provides separate functions for adding, viewing, searching, updating, and deleting student records. It also provides automatic grade calculation based on student marks.

For data persistence, student records are stored in a JSON file. This allows the application to load previously stored records when it starts and save changes whenever student information is modified.

A performance analysis module has also been implemented using **Pandas and NumPy**. Pandas is used to organize student records into a DataFrame and perform operations such as filtering and sorting, while NumPy is used for numerical calculations such as mean, median, minimum, maximum, and standard deviation.

The proposed system therefore combines basic student record management with academic performance analysis in a single lightweight application.

4.3 Value Addition

The major value additions of the proposed system are:

- Simple menu-driven operation.
- Input validation to reduce incorrect data entry.
- Individual-field student updates.
- Persistent storage using JSON.
- Automatic grade calculation.
- Student performance analysis using Pandas and NumPy.
- Sorting and filtering of student performance data.
- Basic exception handling.
- Easy deployment without requiring a database server.

4.4 Code Submission

The complete source code of the Student Management System has been uploaded to the GitHub repository created for the internship.

GitHub Code Link:

[LINK TO BE ADDED AFTER FINAL REPOSITORY UPDATE]

4.5 Report Submission

The final internship report will be uploaded to the same GitHub repository along with the project source code.

GitHub Report Link:

[LINK TO BE ADDED AFTER FINAL PDF UPLOAD]

5. Proposed Design / Model

The proposed Student Management System follows a simple modular design. The application is developed using Python and operates through a console-based menu. Each major operation is implemented using a separate function, making the program easier to understand, test, and maintain.

The system accepts input from the user through the main menu and performs the selected operation. Student records are maintained using Python data structures and are stored permanently in a JSON file. For academic performance analysis, the stored records are converted into a Pandas DataFrame and NumPy is used for numerical calculations.

The overall system consists of four major components:

1. **User Interface and Menu**
2. **Student Record Management**
3. **Data Storage**
4. **Performance Analysis**

5.1 High Level Diagram

The high-level architecture of the Student Management System is shown below. The user interacts with the Python application through the console menu. Depending on the selected operation, the system performs student record management, grade calculation, or performance analysis.

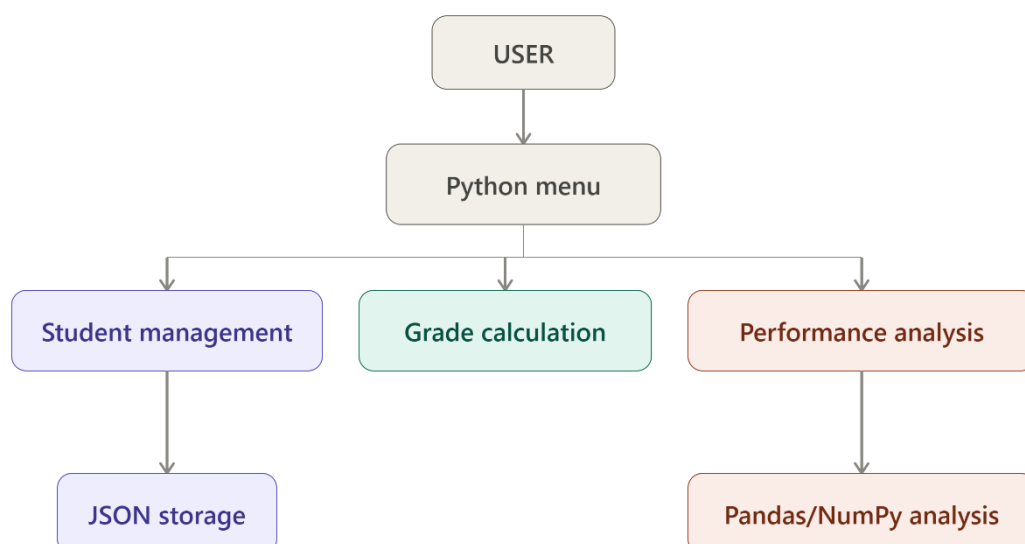


Figure 1 : High-Level Architecture of the Student Management System

5.2 Low Level Diagram

The low-level design describes the main functional modules of the application and the flow of information between them.

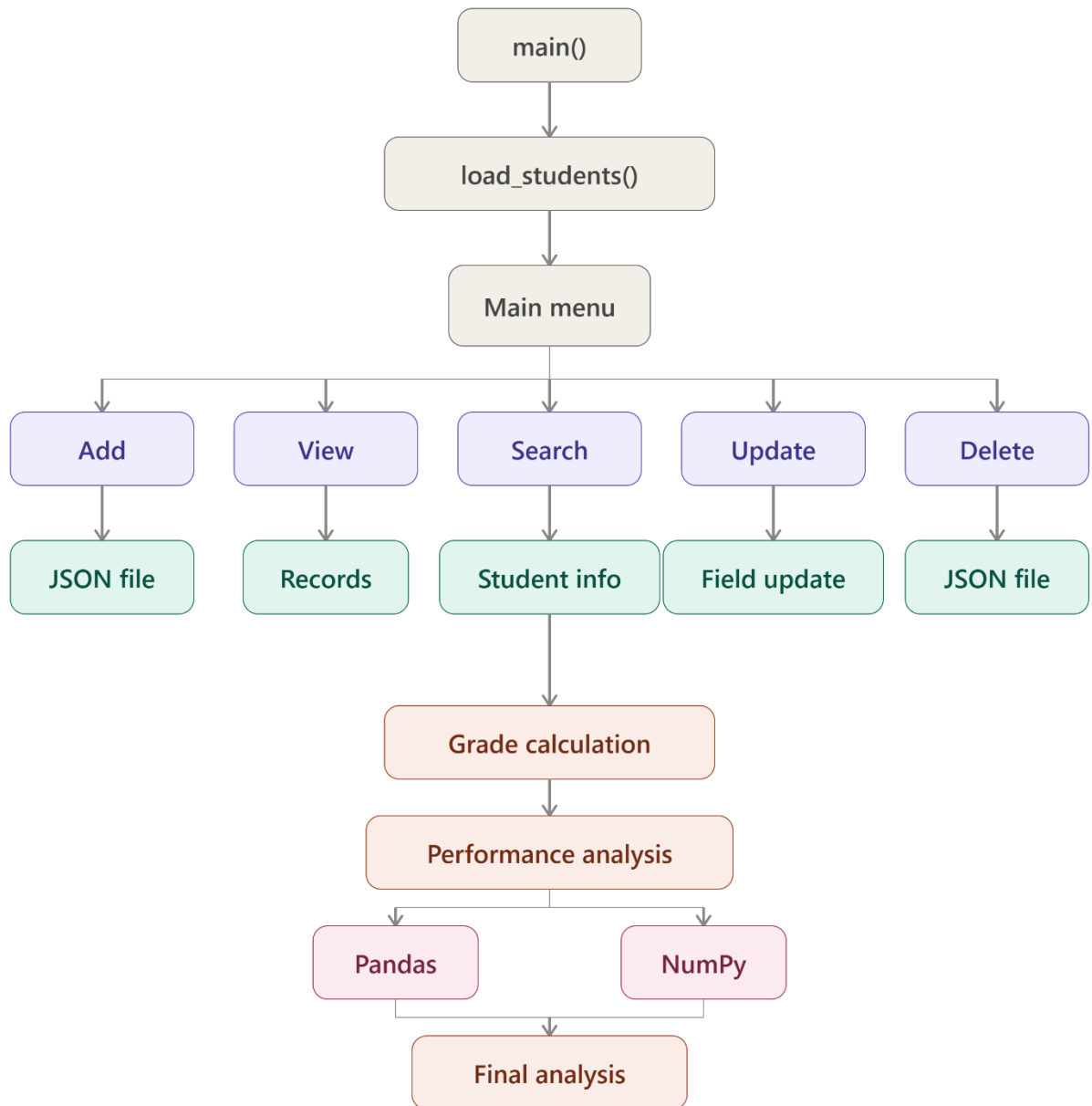


Figure 2 : Low-Level Functional Flow of the Student Management System

5.3 Interfaces

The system uses a command-line interface (CLI) through which users interact with the application. The main menu provides access to all available operations.

The interface is designed to be simple and beginner-friendly. Users select an operation by entering the corresponding menu number and then provide the required information.

The major interfaces of the system are:

- Add Student
- View All Students
- Search Student
- Update Student
- Delete Student
- Calculate Grade
- Performance Analysis
- Exit

```
=====
                        STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Calculate Grade
7. Performance Analysis
8. Exit
=====
Enter your choice: |
```

Figure 3: Main Menu of the Student Management System

Enter your choice: 1

===== ADD STUDENT =====

Enter Student ID: 101
 Enter Student Name: Rahul
 Enter Student Age: 19
 Enter Course: Btech-Cse
 Enter Year: 2
 Enter Marks: 90

Student added successfully!

Figure 4: Adding a Student

Enter your choice: 2

===== ALL STUDENTS =====

Student ID : 123
 Name : wdsd
 Age : 12
 Course : csdsd
 Year : 12311
 Marks : 1213323.0

Student ID : 101
 Name : Rahul
 Age : 19
 Course : Btech-Cse
 Year : 2
 Marks : 90.0

Student ID : 102
 Name : Ramesh
 Age : 20
 Course : Btech-AiMl
 Year : 1
 Marks : 75.0

Student ID : 103
 Name : Priya
 Age : 19
 Course : Btech-Cse
 Year : 2
 Marks : 40.0

Figure 5 : Viewing Existing Student

5.4 Student Record Structure

Each student is represented as a Python dictionary containing the student's basic academic information.

Field	Description
ID	Unique identification number of the student
Name	Student's name
Age	Student's age
Course	Student's course
Year	Current academic year
Marks	Marks obtained by the student

Example :

```
{  
    "id": 101,  
    "name": "Rahul",  
    "age": 19,  
    "course": "B.Tech CSE",  
    "year": 2,  
    "marks": 85  
}
```

Enter your choice: 1

===== ADD STUDENT =====

Enter Student ID: 101

Enter Student Name: Rahul

Enter Student Age: 19

Enter Course: Btech-Cse

Enter Year: 2

Enter Marks: 90

Student added successfully!

Figure 6: Displaying Student Records

5.5 Data Storage

The system uses JSON (JavaScript Object Notation) for persistent storage of student records. The `students.json` file stores the student information in a structured format.

When the application starts, the `load_students()` function reads the existing JSON file and loads the records into the Python program. Whenever a student is added, updated, or deleted, the `save_students()` function writes the updated records back to the JSON file.

This approach allows student records to remain available even after the application is closed.

5.6 Performance Analysis Module

The Performance Analysis module was developed to apply the data-processing concepts learned during the internship. The module uses Pandas and NumPy to analyze student marks.

First, the student records are converted into a Pandas DataFrame. This provides a structured tabular representation of the student information and makes operations such as filtering and sorting easier.

The marks are then converted into a NumPy array for numerical calculations. NumPy functions are used to calculate statistical values such as mean, median, minimum, maximum, and standard deviation.

The module also identifies the number of passed and failed students and sorts students according to their marks.

Technology	Operation
Pandas	Create DataFrame
Pandas	Filter student records
Pandas	Sort students by marks
Pandas	Count passed/failed students
NumPy	Calculate mean
NumPy	Calculate median
NumPy	Calculate minimum
NumPy	Calculate maximum
NumPy	Calculate standard deviation

Enter your choice: 7

===== PERFORMANCE ANALYSIS =====

Student Performance Table:

id	name	course	marks
123	wdsd	csdsd	1213323.0
101	Rahul	Btech-Cse	90.0
102	Ramesh	Btech-AiMl	72.0
103	Priya	Btech-Cse	40.0

===== STATISTICS =====

Total Students	: 4
Average Marks	: 303381.25
Median Marks	: 81.0
Highest Marks	: 1213323.0
Lowest Marks	: 40.0
Standard Deviation	: 525355.11
Passed Students	: 3
Failed Students	: 1

Students Sorted by Marks:

id	name	marks
123	wdsd	1213323.0
101	Rahul	90.0
102	Ramesh	72.0
103	Priya	40.0

Figure 6: Student Performance Analysis Using NumPy and Pandas

5.7 Grade Calculation

The system provides automatic grade calculation based on the marks obtained by a student. Conditional statements are used to assign a grade according to predefined marks ranges.

Marks	Grade
90–100	A+
80–89	A
70–79	B
60–69	C
50–59	D
Below 50	F

This feature demonstrates the use of if, elif, and else conditional statements in Python.

```
Enter your choice: 6

===== CALCULATE GRADE =====
Enter Student ID: 102

Student Name : Ramesh
Marks       : 72.0
Grade       : B
```

Figure 7: Grade Calculation

5.8 Input Validation and Exception Handling

Input validation has been implemented to ensure that users enter valid information. For example, Student ID must be a positive number, age must fall within the specified range, academic year must be between 1 and 4, and marks must be between 0 and 100.

Exception handling is also used when numerical input is entered incorrectly. The program catches `ValueError` and asks the user to enter a valid value instead of terminating unexpectedly.

These mechanisms improve the reliability and user-friendliness of the application.

6. Performance Test

The Student Management System was tested to verify that its major functionalities work correctly and that the application responds appropriately to valid and invalid user inputs.

Testing was performed by executing the application through different menu options and providing different types of input. The testing focused on student record management, data persistence, grade calculation, input validation, and performance analysis using Pandas and NumPy.

The main objective of testing was to ensure that the system performs its intended operations correctly without unexpected termination.

6.1 Test Plan / Test Cases

The following test cases were used to verify the major functionalities of the system.

Test Case	Operation	Input / Condition	Expected Result
TC01	Add Student	Enter valid student details	Student record is added successfully
TC02	Add Student	Enter an existing Student ID	System displays duplicate ID message
TC03	View Students	Student records are available	All stored records are displayed
TC04	Search Student	Enter an existing Student ID	Correct student record is displayed
TC05	Search Student	Enter a non-existing ID	"Student not found" message is displayed
TC06	Update Student	Select a valid student and field	Selected information is updated
TC07	Delete Student	Enter an existing Student ID	Student record is deleted
TC08	Calculate Grade	Enter an existing Student ID	Correct grade is displayed
TC09	Invalid Input	Enter text instead of a number	System displays an error message
TC10	Marks Validation	Enter marks outside 0–100	System rejects the input
TC11	Performance Analysis	Multiple student records available	Statistical analysis is displayed
TC12	Data Persistence	Restart the application	Previously saved records are loaded

6.2 Test Procedure

The application was executed from the command line and each major functionality was tested individually.

First, multiple student records were created using the Add Student option. The View Students and Search Student options were then used to verify that the records were stored and displayed correctly.

The Update Student functionality was tested by modifying individual fields of an existing student record. The Delete Student functionality was then tested by removing a selected record.

The grade calculation feature was tested using different marks to verify that the appropriate grade was assigned according to the defined grading criteria.

Invalid inputs were also provided to test the input validation and exception handling mechanisms. The application correctly handled invalid numerical input and values outside the allowed ranges.

Finally, the Performance Analysis option was tested using multiple student records. Pandas was used to organize the records into a DataFrame, while NumPy was used for numerical calculations such as mean, median, minimum, maximum, and standard deviation.

The application was also restarted after modifying student records to verify that the JSON-based data storage maintained the records between program executions.

6.3 Performance Outcome

The testing results showed that the major functionalities of the Student Management System operated as expected.

The application successfully performed student record creation, viewing, searching, updating, and deletion. The selective update functionality reduced unnecessary data entry by allowing users to modify only the required field.

The grade calculation module correctly assigned grades according to the defined marks ranges. Input validation prevented invalid values from being accepted, while exception handling prevented the program from terminating unexpectedly due to incorrect numerical input.

The JSON storage mechanism successfully maintained student records between program executions. The Pandas and NumPy based performance analysis also successfully processed the available student records and generated the required statistical information.

Overall, the testing demonstrated that the application meets the functional requirements defined for the project and provides a reliable basic solution for student record management and academic performance analysis.

7. My Learnings

The four-week Python Learning Internship provided practical exposure to Python programming and helped in developing a better understanding of programming concepts through practical implementation.

During the internship, I learned the fundamentals of Python, including variables, data types, conditional statements, loops, functions, lists, dictionaries, exception handling, and file handling. These concepts were applied while developing the Student Management System.

I also gained an introduction to NumPy and Pandas. NumPy helped me understand numerical operations and statistical calculations, while Pandas provided an efficient way to organize and process student information in a tabular format.

During the development of the project, I learned how to divide a program into multiple functions according to their responsibilities. I also gained experience in validating user input and handling errors so that the application could continue running without unexpected termination.

The project provided practical experience with JSON-based file storage. I learned how data can be saved to a file and loaded again when the application is restarted, providing basic persistent storage without requiring a database.

I also gained experience with Git and GitHub for maintaining project versions and managing the source code. Working with separate feature branches and merging completed features into the main branch helped me understand basic version control practices.

Overall, the internship improved my Python programming, problem-solving, debugging, data-handling, and software development skills. The experience also provided a foundation for learning more advanced Python applications and data-related technologies in the future.

8. Future Work Scope

The current Student Management System provides the basic functionality required for managing student records and analyzing academic performance. However, several improvements can be made in the future to make the system more powerful and suitable for larger-scale use.

Possible future enhancements include:

1. **Graphical User Interface:**
A graphical interface can be developed using technologies such as Tkinter, PyQt, or another Python GUI framework to make the system easier to use.
2. **Database Integration:**
JSON-based storage can be replaced or supplemented with a database such as SQLite or MySQL for handling a larger number of records.
3. **Subject-Wise Marks:**
The system can be extended to store marks for individual subjects and calculate subject-wise and overall performance.
4. **Attendance Management:**
Attendance records can be added to provide a more complete student management solution.
5. **Report Generation:**
The system can generate student performance reports in formats such as PDF or Excel.
6. **Data Visualization:**
Charts and graphs can be added to visually represent student performance and class statistics.
7. **User Authentication:**
Login functionality can be introduced to restrict access to authorized users.
8. **Web-Based Application:**
The console application can eventually be converted into a web application using Python frameworks such as Flask or Django.

These improvements could make the system more scalable, user-friendly, and suitable for practical educational environments.